1 - Dashboard, Profile & Pagination

Now we're going to start working on the dashboard, which will have 3 different things on it. It will have the user's job listings with buttons to edit and delete. It will have a form to edit the profile info including name, email and avatar. Then later, when we add application submissions, it will have the applicants under each listing. So we'll start on the user listings first.

Another thing that I want to do in this section is add pagination on the jobs page. If we have 500 listings, we obviously don't want to fetch or show all 500 at once. That would be horrible for performance. So we will implement pagination as well as customize the look of it.

2 - Dashboard Controller and View

The dashboard page will have a form with the user's name and email. The user can update their name and email from this form by submitting to the profile controller method, which we will update soon. It will also have the user's job listings and any applicant submissions to those job listings.

Dashboard Controller

Let's create a new controller for the dashboard:

```
php artisan make:controller DashboardController
```

Add an index method to the controller:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use App\Models\Job;
use Illuminate\View\View;

class DashboardController extends Controller
{
    public function index(Request $request): View
    {
        // Get the authenticated user
        $user = Auth::user();

        // Get all job listings for the authenticated user
        $jobs = Job::where('user_id', $user->id)->get();

        return view('dashboard', compact('user', 'jobs'));
    }
```

```
}
```

We get the user and all job listings for the authenticated user. We then pass the user and jobs to the view.

Dashboard Route

Let's add our route. Open the `routes/web.php` file and add the following import:

```
use App\Http\Controllers\DashboardController;
```

Add the route and apply the `auth` middleware:

```
Route::get('/dashboard', [DashboardController::class, 'index'])-
>name('dashboard')->middleware('auth');
```

Profile View

Create a new view in the `resources/views/dashboard` directory called `index.blade.php`. Add the following content:

```
<x-layout> Dashboard </x-layout>
```

Make sure that the page shows when you go to `/dashboard`. There should already be a link to the dashboard page in the navigation bar.

In the next lesson, we will add the form to update the user's name and email.

3 - Dashboard User Job Listings

Now we want the user's listings to be displayed on their dashboard page. We already have everything in place. In the controller, we are passing the jobs into the view.

Add the following to the dashboard view:

```
<div class="bg-white p-8 rounded-lg shadow-md w-full">
  <h3 class="text-3xl text-center font-bold mb-4">My Job Listings</h3>
  @forelse ($jobs as $job)
  <div
    class="flex justify-between items-center border-b-2 border-gray-200 py-2"
  >
    <div>
      <h3 class="text-xl font-semibold">{{ $job->title }}</h3>
```

```
        <p class="text-gray-700">{{ $job->job_type }}</p>
      </div>
     <div class="flex space-x-4">
        <a
          href="{{ route('jobs.edit', $job->id) }}"
          class="bg-blue-500 hover:bg-blue-600 text-white px-4 py-2 rounded text-sm"
          >Edit</a
        >
        <form
          method="POST"
          action="{{ route('jobs.destroy', $job->id) }}"
          onsubmit="return confirm('Are you sure you want to delete this job?');"
        >
          @csrf @method('DELETE')
          <button
            type="submit"
            class="bg-red-500 hover:bg-red-600 text-white px-4 py-2 rounded text-sm"
          >
            Delete
          </button>
        </form>
      </div>
    </div>
    @empty
    <p class="text-gray-700">You have no job listings.</p>
    @endforelse
  </div>
```

Control Delete Redirect

Right now the jobs are showing fine and you can edit and delete them using the buttons. However right now when you delete a job from the dashboard page, you are redirected to the homepage. I don't want that. I want to stay on the dashboard page if we delete from this page.

First, we need to add a query string to the delete form action. Open the `resources/views/dashboard/index.blade.php` file and add the following to the delete form:

```
<form method="POST" action="{{ route('jobs.destroy', $job->id) }}?from=dashboard"
onsubmit="return confirm('Are you sure you want to delete this job?');">
```

We added `?from=dashboard` to the end of the route. This will add a query string to the URL when the form is submitted.

Open the `app/Http/Controllers/JobController.php` file and edit the `destroy` method by adding this line right above the redirect that is already there:

```
    // Check if the request came from the dashboard page
    if (request()->query('from') === 'dashboard') {
        return redirect()->route('dashboard.index')->with('success', 'Job listing
```

```
deleted successfully!');
}
```

Now when you delete a job from the dashboard page, you will stay on that page.

You can run `php artisan db:seed` to get the listings back if you deleted them.

4 - Profile Controller & Update User Info

We need to add a form on the dashboard page that shows the user's info and we want to be able to update that info. I am going to create a separate controller for this because it relates more to the user "profile".

Let's open the `/resources/views/dashboard/index.blade.php`. I want the profile form and the listings to be side by side. So add this at the very top just after the opening `<x-layout>`:

```
<section class="flex flex-col md:flex-row gap-6">
```

At the bottom, close it just above the closing `</x-layout>`. I am also going to show the bottom banner component on the dashboard:

```
  </section>
  <x-bottom-banner />
</x-layout>
```

Now we need to add the following above the job listings div:

```
<div class="bg-white p-8 rounded-lg shadow-md w-full md:w-1/2">
  <h3 class="text-3xl text-center font-bold mb-4">Profile Info</h3>
  <form method="POST" action="{{ route('profile.update') }}"
enctype="multipart/form-data">
    @csrf
    @method('PUT')

    <div class="mb-4">
      <label class="block text-gray-700" for="name">Name</label>
      <input id="name" type="text" name="name" value="{{ $user->name }}"
        class="w-full px-4 py-2 border rounded focus:outline-none" />
    </div>
    <div class="mb-4">
      <label class="block text-gray-700" for="email">Email</label>
      <input id="email" type="text" name="email" value="{{ $user->email }}"
        class="w-full px-4 py-2 border rounded focus:outline-none" />
    </div>
    <div class="mb-4">
      <label class="block text-gray-700" for="avatar">Profile Avatar</label>
      <input id="avatar" type="file" name="avatar" class="w-full px-4 py-2 border
```

```
rounded focus:outline-none" />
    </div>
    <button type="submit"
      class="w-full bg-green-500 hover:bg-green-600 text-white px-4 py-2 my-3
rounded focus:outline-none">Save</button>
  </form>
</div>
```

Profile Controller

Let's create a new profile controller:

```
php artisan make:controller ProfileController
```

Open the controller and add the import for the user model:

```
use App\Models\User;
```

Next, let's add the `update` method to the `ProfileController`:

```
public function update(Request $request): RedirectResponse
{
    // Get the authenticated user
    $user = Auth::user();

    // Validate the incoming request data
    $validatedData = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|string|email|max:255|unique:users,email,' . $user-
>id,
    ]);

    // Update the user's information
    $user->update($validatedData);

    // Redirect back to the dashboard page with a success message
    return redirect()->route('dashboard.show')->with('success', 'User info updated
successfully!');
}
```

Be sure to import the `RedirectResponse` class at the top of the file:

```
use Illuminate\Http\RedirectResponse;
```

This is pretty self-explanatory. We are validating the incoming request data, updating the user's information, and then redirecting back to the profile page with a success message.

The reason we pass the `$user->id` to the `email` validation rule is to make sure that the email is unique, except for the current user. This is because we don't want to allow the user to change their email to one that already exists in the database.

Add Update Route

You need to create the update route. Open the `routes/web.php` file and add the following import:

```
use App\Http\Controllers\ProfileController;
```

Now add the route:

```
Route::put('/profile', [ProfileController::class, 'update'])-
>name('profile.update')->middleware('auth');
```

Now you should be able to update your name and email and it should show in the dashboard.

5 - User Avatar

I decided that I want to have users have the ability to upload an avatar. Right now, from the dashboard, we can see the user's name and email. I want to add an avatar to this as well. Let's add the avatar upload to the form.

New Migration

We need to add a new column to the `users` table to store the avatar. Let's create a new migration for this.

```
php artisan make:migration add_avatar_to_users_table --table=users
```

Open the migration file and add the following code to the `up` method:

```
Schema::table('users', function (Blueprint $table) {
    $table->string('avatar')->nullable()->after('email');
});
```

Add the following code to the `down` method:

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn('avatar');
```

```
});
```

Run the migration:

```
php artisan migrate
```

Update User Model

Open the `app/Models/User.php` file and add the new field to the `$fillable` array:

```
protected $fillable = [
    'name',
    'email',
    'password',
    'avatar',
];
```

Update the Form

Open the `resources/views/dashboard/show.blade.php` file and add the following code right under the email field:

```
<div class="mb-4">
  <label class="block text-gray-700" for="avatar">Profile Avatar</label>
  <input
    id="avatar"
    type="file"
    name="avatar"
    class="w-full px-4 py-2 border rounded focus:outline-none"
  />
</div>
```

Avatar Preview

Let's add a preview of the avatar. Add the following right under the `h3` tag at the top of the page:

```
@if($user->avatar)
<div class="mt-2 flex justify-center">
  <img
    src="{{ asset('storage/' . $user->avatar) }}"
    alt="Avatar"
    class="w-32 h-32 object-cover rounded-full"
  />
```

```
    </div>
    @endif
```

Update the Profile Controller

Now open the `app/Http/Controllers/ProfileController.php` file and add the following code to the `update` method:

```
    public function update(Request $request)
        {
            $user = Auth::user();

            // Validate the request
            $request->validate([
                'name' => 'required|string|max:255',
                'email' => 'required|email|max:255',
                'avatar' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
            ]);

            // Update user details
            $user->name = $request->input('name');
            $user->email = $request->input('email');

            // Handle file upload
            if ($request->hasFile('avatar')) {
                // Delete old avatar if exists
                if ($user->avatar) {
                    Storage::delete('public/' . $user->avatar);
                }

                // Store new avatar
                $avatarPath = $request->file('avatar')->store('avatars', 'public');
                $user->avatar = $avatarPath;
            }

            $user->save();

            return redirect()->route('dashboard.show')->with('success', 'Profile
    updated successfully.');
        }
```

Your file should be uploaded to `/storage/app/public/avatars` and should display in the preview. In the next lesson, let's show it in the navbar.

6 - Show Avatar in Header

We are going to show the user's avatar in the header. I also want to show a default avatar if the user does not have one.

There is an image in the downloads for this lesson called `default-avatar.png`. You can use this image as the default avatar. You can use any image you want just rename it to `default-avatar.png`. Put this image in the `/storage/app/public/avatars/` directory.

Update The Header Component

Open the `resources/views/components/header.blade.php` file and add this right under the "Create job" link. It will be right before the `@else` which is from the if statement that checks if the user is logged in.

```
<!-- User Avatar -->
<div class="flex items-center space-x-3">
  @if(Auth::user()->avatar)
  <img
    src="{{ asset('storage/' . Auth::user()->avatar) }}"
    alt="{{ Auth::user()->name }}"
    class="w-10 h-10 rounded-full"
  />
  @else
  <img
    src="{{ asset('storage/avatars/default-avatar.png') }}"
    alt="{{ Auth::user()->name }}"
    class="w-10 h-10 rounded-full"
  />
  @endif
</div>
```

Now the user's avatar will show in the navbar. If the user does not have an avatar, the default avatar will show.

Rearrange The Navbar

This is 100% up to you but I am going to rearrange the navbar a bit. I am going to get rid of the dashboard link and make the avatar the link to the dashboard. I am also going to move the logout link to the end.

Here is the final version of the `resources/views/components/header.blade.php` file.

```
<header class="bg-blue-900 text-white p-4" x-data="{ open: false }">
  <div class="container mx-auto flex justify-between items-center">
    <h1 class="text-3xl font-semibold">
      <a href="{{ route('home') }}">Workopia</a>
    </h1>
    <nav class="hidden md:flex items-center space-x-4">
      <x-nav-link url="/" :active="request()->is('/')">Home</x-nav-link>
      <x-nav-link url="/jobs" :active="request()->is('jobs')"
        >All Jobs</x-nav-link
      >
      @auth
      <x-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')"
        >Saved Jobs</x-nav-link
      >
```

```
        <x-button-link url="/jobs/create" type="button" icon="edit"
          >Create Job</x-button-link
        >

        <!-- User Avatar -->
        <div class="flex items-center space-x-3">
          <a href="{{ route('dashboard.show') }}">
            @if(Auth::user()->avatar)
            <img
              src="{{ asset('storage/' . Auth::user()->avatar) }}"
              alt="{{ Auth::user()->name }}"
              class="w-10 h-10 rounded-full"
            />
            @else
            <img
              src="{{ asset('storage/avatars/default-avatar.png') }}"
              alt="{{ Auth::user()->name }}"
              class="w-10 h-10 rounded-full"
            />
            @endif
          </a>
        </div>
        <form method="POST" action="{{ route('logout') }}">
          @csrf
          <button type="submit" class="text-white">
            <i class="fa fa-sign-out"></i> Logout
          </button>
        </form>
        @else
        <x-nav-link url="/login" :active="request()->is('login')"
          >Login</x-nav-link
        >
        <x-nav-link url="/register" :active="request()->is('register')"
          >Register</x-nav-link
        >
        @endauth
      </nav>
      <button
        @click="open = !open"
        class="text-white md:hidden flex items-center"
      >
        <i class="fa fa-bars text-2xl"></i>
      </button>
    </div>
    <!-- Mobile Menu -->
    <nav
      x-show="open"
      @click.away="open = false"
      class="md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
    >
      <x-mobile-nav-link url="/" :active="request()->is('/')"
        >Home</x-mobile-nav-link
      >
      <x-mobile-nav-link url="/jobs" :active="request()->is('jobs')"
```

```
        >All Jobs</x-mobile-nav-link
      >
      @auth
      <x-mobile-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')"
        >Saved Jobs</x-mobile-nav-link
      >
      <x-mobile-nav-link
        url="/dashboard"
        :active="request()->is('dashboard')"
        icon="gauge"
        >Dashboard
      </x-mobile-nav-link>
      <form method="POST" action="{{ route('logout') }}">
        @csrf
        <button type="submit" class="text-white">
          <i class="fa fa-sign-out"></i> Logout
        </button>
      </form>
      <div class="py-2">
        <x-button-link url="/jobs/create" type="button" icon="edit"
          >Create Job</x-button-link
        >
      </div>
      @else
      <x-mobile-nav-link url="/login" :active="request()->is('login')"
        >Login</x-mobile-nav-link
      >
      <x-mobile-nav-link url="/register" :active="request()->is('register')"
        >Register</x-mobile-nav-link
      >
      @endauth
    </nav>
  </header>
```

7 - Simple Job Pagination

Right now, all jobs will be displayed at once even if there are hundreds of them. This is not a good idea. We should paginate them. Laravel has a built-in pagination system. We will use that to paginate the jobs.

Job Controller

The first step is to edit the `index` method in the `JobController` to paginate the jobs. Change the following code:

```
$jobs = Job::all();
```

To this:

```
$jobs = Job::paginate(3);
```

I am only using `3` to test it out. I will change it to a higher number later.

If you go to the `/jobs` route, you will only see three listings. If you manually type in `/jobs?page=2`, you will see the next three listings.

Add Links

Now let's add the links. Open the `resources/views/jobs/index.blade.php` file and add the following code right above the closing `</x-layout>`:

```
<!-- Pagination Links -->
<div class="mt-4">{{ $jobs->links() }}</div>
```

It's as simple as that to add pagination in Laravel.

One issue though is that you are bound to the style of the pagination links. We also only have prev and next and not the individual page numbers. We can change this by publishing the pagination view and customizing it. I will show you how to do that in the next lesson.

8 - Customizing The Pagination View

Right now we have a prev and next button, but what if we want to style it differently and add individual page numbers? Right now, that code is not available to us in our views. However we can publish the pagination view and customize it.

Open a terminal and run the following command:

```
artisan vendor:publish --tag=laravel-pagination
```

This will publish the pagination view into the `resources/views/vendor/pagination` directory. There will be multiple files in there and it depends on what CSS framework you are using. I am using Tailwind CSS, so I will edit the `tailwind.blade.php` file.

Open the `tailwind.blade.php` file and wipe away all the code and add the following:

```
@if ($paginator->hasPages())
<nav
  role="navigation"
  aria-label="Pagination Navigation"
  class="flex justify-center"
>
  {{-- Previous Page Link --}} @if ($paginator->onFirstPage())
```

```
    <span class="px-4 py-2 bg-gray-300 text-gray-500 rounded-l-lg">Previous</span>
    @else
    <a
      href="{{ $paginator->previousPageUrl() }}"
      rel="prev"
      class="px-4 py-2 bg-blue-500 text-white rounded-l-lg hover:bg-blue-600"
      >Previous</a
    >
    @endif {{-- Pagination Elements --}}
    @foreach ($elements as $element)
      {{--"Three Dots" Separator --}}
      @if (is_string($element))
          <span class="px-4 py-2 bg-gray-300 text-gray-500">{{ $element }}</span>
      @endif
      {{-- Array Of Links --}}
      @if (is_array($element))
        @foreach ($element as $page => $url)
          @if ($page == $paginator->currentPage())
            <span class="px-4 py-2 bg-blue-500 text-white ">{{ $page }}</span>
          @else
            <a
              href="{{ $url }}"
              class="px-4 py-2 bg-gray-200 text-gray-700 hover:bg-blue-600
  hover:text-white"
              >{{ $page }}</a
            >
          @endif
        @endforeach
      @endif
    @endforeach
    {{-- Next Page Link --}}
    @if( $paginator->hasMorePages())
      <a
        href="{{ $paginator->nextPageUrl() }}"
        rel="next"
        class="px-4 py-2 bg-blue-500 text-white rounded-r-lg hover:bg-blue-600"
        >Next</a
      >
    @else
      <span class="px-4 py-2 bg-gray-300 text-gray-500 rounded-r-lg">Next</span>
    @endif
  </nav>
  @endif
```

We are using the same HTML and styles from our template. We are just making it dynamic by using loops, certain directives and variables.

You should now see the new pagination with the individual page numbers. You can customize it further by adding more classes or changing the styles.

Let's change the number of listings from 3 to 12 now that we are done with the pagination. Open the `app/Http/Controllers/JobController.php` file edit the line of code in the `index` method:

```
$jobs = Job::paginate(12);
```

Now if you have under 12 listings, you won't even see the pagination links.